



Sudan University for Science and Technology
College of Computer Science and
Information Technology

A Platform for Building Domain-Specific
Conversational AI Agents

منصة لبناء وكلاء ذكاء إصطناعي تخاطبي متخصصين في
مجالات محددة

محراب الطاهر محمد السبكي - هندسة البرمجيات

ربي جمال مأمون عوض - تقانة المعلومات

أبوبكر أسامة محمد سليمان - علوم حاسوب

المشرف: د/محمد عوض

Submission Date: June 2026

Dedication

We dedicate this work to our parents for their love, support, and sacrifices throughout our lives. We also dedicate this work to our families, teachers, and everyone who supported us during our academic journey. A special dedication from Mihrab: To my dear brother, Musab, may Allah have mercy on him. He will always remain in my heart and memories.

I pray that Allah grants him Paradise and reunites us in the highest level of Jannah.

Acknowledgements

First of all, we thank Allah for giving us the strength and patience to complete this project. We would like to honestly thank our supervisor, Dr. Mohamed Awad, for his guidance, support, and helpful advice throughout this project. We also thank the faculty members and staff of the department for their help and support during our studies. We are also grateful to our friends and colleagues for their cooperation and support during the completion of this work. Finally, we would like to thank everyone who contributed, directly or indirectly, to the success of this project.

Abstract

Despite recent developments in conversational Artificial Intelligence (AI), building custom AI agents remains inaccessible to many organizations and individuals due to the technical expertise and high costs required by many of the current frameworks and tools. This project presents the design and implementation of an open-source, multi-tenant platform for building domain-specific conversational AI agents.

The platform exposes its functionality through a RESTful API built with FastAPI allowing developers to extend the platform and integrate agents' capabilities into any application. It also provides a web-based admin panel built with React to enable users to configure and manage agents. The core of an agent is a Retrieval-Augmented Generation (RAG) pipeline that ingests, chunks, embeds, and stores document vector embeddings in the Qdrant vector database, along with GPT-5-mini Large Language Model (LLM) serving as the brain for generating responses. Agents also support external tool use, enabling integration with third party APIs.

The platform was evaluated across three aspects. Functional evaluation confirmed that all features behaved as expected through black-box testing. A usability evaluation using the System Usability Scale (SUS) across ten participants resulted in a score of 83.5 indicating good usability. An AI core evaluation using the RAGAS framework for the RAG pipeline produced a faithfulness score of 93.3% and a context recall of 89.4%, while metrics adapted from API-Bank were used for tool use evaluation, scoring 88% tool selection accuracy and 100% parameter and result integration accuracy.

The results demonstrate that the platform effectively makes AI agent creation accessible to both technical and non-technical users, and provides an extensible open-source foundation for further development.

المستخلص

على الرغم من التطورات في مجال الذكاء الاصطناعي التخليقي، لا يزال بناء وكلاء مخصصين غير متاح للعديد من الأفراد و الجهات نظرا للخبرة التقنية و التكاليف المادية العالية التي تتطلبها العديد من الأدوات الحالية. يقدم هذا المشروع تصميم و تطبيق منصة مفتوحة المصدر لبناء وكلاء ذكاء اصطناعي تخليقي متخصصين في مجالات محددة.

تعرض المنصة وظائفها من خلال RESTful API مبنية باستخدام FastAPI مما يتيح للمطورين توسيع المنصة و دمج وكلائهم في أي تطبيق، كما توفر لوحة تحكم إدارية مبنية باستخدام React تمكن المستخدمين من إدارة وكلائهم. الأساس لكل وكيل هو خط ال RAG الذي يقوم بتقسيم النص إلى أجزاء أصغر ثم تحويلها إلى متجهات ثم تخزينها في قاعدة بيانات Qdrant. إلى جانب الخط. يتم استخدام النموذج اللغوي الكبير gpt-5-mini لتوليد الإجابات. كما يدعم الوكلاء استخدام الأدوات الخارجية (tool use) مما يتيح التكامل والربط مع ال APIs الخارجية.

جرى تقييم المنصة من ثلاثة جوانب. أكد التقييم الوظيفي بأن جميع خواص المنصة تعمل كما هو متوقع من خلال اختبار black-box. كما تم تقييم سهولة الاستخدام بمقياس SUS على عشرة مشاركين وأسفر عن نتيجة 83.5 نقطة مما يدل على سهولة الاستخدام. أما التقييم الثالث كان لقياس أداء الوكيل من ناحيتين: الأولى هي ال RAG وتم قياسها باستخدام إطار عمل RAGAS وقد حقق نسبة التزام بالمصدر بلغت 93.3% ومعدل إسترجاع سياق بلغ 89.4%، والثانية هي قدرة استخدام الأدوات الخارجية بمقاييس مستوحاة من API-Bank وقد حققت دقة اختيار الأداة نسبة بلغت 88% ودقة اختيار المعلومات ودمج النتائج بلغت 100%.

تثبت النتائج أن المنصة تسهل من إنشاء وكلاء الذكاء الاصطناعي لغير التقنيين، وتوفر أساسا مفتوح المصدر وقابل للتوسعة من قبل المبرمجين.

Table of Contents

CHAPTER ONE: INTRODUCTION.....	1
1.1 Background	2
1.2 Importance of the Project.....	2
1.3 Problem Statement	2
1.4 Objectives.....	3
1.5 Scope	3
1.6 Methodology	4
CHAPTER TWO: THEORETICAL FRAMEWORK AND LITERATURE REVIEW.....	5
2.1 Introduction	6
2.2 Theoretical Framework	6
2.2.1 Large Language Models (LLMs)	6
2.2.2 Retrieval-Augmented Generation (RAG)	6
2.2.3 Embeddings, Vector Databases, and Semantic Search	7
2.2.3.1 Embeddings.....	7
2.2.3.2 Vector Databases	7
2.2.3.3 Semantic Search.....	7
2.2.4 Tool Use	8
2.2.5 Multi-tenancy Architecture	8
2.3 Literature Review.....	8
2.3.1 Agents: An Open-source Framework for Autonomous Language Agents	8
2.3.2 Design and Implementation of an AI-Based Chatbot Framework with Retrieval-Augmented Generation and Integrated Recommender System for Interactive User Support.....	9
2.3.3 FACTS About Building Retrieval Augmented Generation-based Chatbots...	9
2.3.4 Rasa Pro Framework	9
2.3.5 Bot Framework SDK.....	10
2.3.6 Summary of Literature Review	10
2.4 Summary	12
CHAPTER THREE: METHODOLOGY	14
3.1 Introduction.....	15
3.2 Development Methodology	15
3.3 Requirements Analysis	15
3.3.1 Functional Requirements.....	15

3.3.2 Non-Functional Requirements	16
3.3.3 Use Case Diagram	17
3.4 System Architecture	18
3.4.1 UI Layer.....	18
3.4.2 API Layer	18
3.4.3 Core Layer	19
3.4.4 Data and External Systems.....	19
3.5 System Design.....	19
3.5.1 Class Diagram	20
3.5.2 Knowledge Ingestion.....	22
3.5.3 Chat.....	23
3.6 Database Design.....	25
3.7 Evaluation Plan	25
3.8 Summary	26
CHAPTER FOUR: IMPLEMENTATION.....	27
4.1 Introduction.....	28
4.2 Tools and Technologies	28
4.2.1 Backend	28
4.2.2 Frontend.....	29
4.2.3 Infrastructure and Tools	29
4.2.4 AI Models.....	29
4.3 Core Layer.....	30
4.3.1 Knowledge Processor	30
4.3.2 Tool Use	30
4.3.3 Chat.....	31
4.4 API Layer	33
4.4.1 Structure	33
4.4.2 Authentication and Authorization	33
4.5 Admin Panel.....	34
4.6 Database	34
4.7 Summary	34
CHAPTER FIVE: RESULTS AND RECOMMENDATIONS.....	35
5.1 Introduction.....	36
5.2 Evaluation and Results.....	36

5.2.1 Functional Evaluation.....	36
5.2.2 Usability Evaluation	39
5.2.3 Core AI Evaluation.....	40
5.2.3.1 Retrieval-Augmented Generation Evaluation.....	41
5.2.3.2 Tool-use Evaluation	43
5.3 Recommendations for Future Work.....	44
REFERENCES	45
APPENDIX A: UI SCREENS	47

List of Tables

Table 2.1 Summary of Literature Review.....	11
Table 4.1 Backend Technologies.....	29
Table 4.2 Frontend Technologies.....	30
Table 4.3 Infrastructure and Tools.....	30
Table 4.4 AI Models.....	31
Table 5.1 Black-box Testing.....	37
Table 5.2 SUS Results.....	40
Table 5.3 RAGAS Results by Configuration.....	42

List of Figures

Figure 3.1 Use Case Diagram.....	17
Figure 3.2 System Architecture.....	18
Figure 3.3 Organizations and Users Class Diagram.....	20
Figure 3.4 Agents Class Diagram.....	21
Figure 3.5 Knowledge Ingestion Activity Diagram.....	22
Figure 3.6 Chat Activity Diagram.....	24
Figure 3.7 Chat Sequence Diagram.....	25
Figure 3.8 ERD Diagram.....	26
Figure 4.1 Agent Execution Pipeline.....	33
Figure A.1 Sign in Page.....	47
Figure A.2 User Registration Page.....	48
Figure A.3 Dashboard Page (when you are not in an organization)	48
Figure A.4 Dashboard Page (when you are in an organization)	49
Figure A.5 Create New Organization Page.....	49
Figure A.6 Organization Configuration Page.....	50
Figure A.7 Invite New Member Page.....	50
Figure A.8 Edit Member Permission Page.....	51
Figure A.9 Remove Team Member Page.....	52
Figure A.10 View Agent Page.....	52
Figure A.11 Create New Agent Page.....	53
Figure A.12 Add Knowledge Base Page.....	53
Figure A.13 Add Tool Page.....	54
Figure A.14 Conversation Logs View Page.....	55
Figure A.15 Agent Playground Page.....	55

Abbreviations

Abbreviations	Term
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
MCP	Model Context Protocol
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
RNN	Recurrent Neural Network
SDK	Software Development Kit
SUS	System Usability Scale
API	Application Programming Interface
FAISS	Facebook AI Similarity Search
NLU	Natural Language Understanding
TF-IDF	Term Frequency–Inverse Document Frequency

CHAPTER ONE: INTRODUCTION

1.1 Background

Conversational AI has evolved at an incredibly fast pace in the last few years, progressing from early rule-based systems to complex Large Language Models (LLMs) (Brown et al. 2020) capable of generating human-like responses. This progress was made possible by the Transformers architecture (Vaswani et al. 2017) which enabled the effective large-scale training of such models. Modern LLM-powered systems leverage advanced techniques to further increase their capabilities such as Retrieval-Augmented Generation (RAG) (Lewis et al. 2020) and tool use (Schick et al. 2023).

However, most existing tools used for building conversational AI agents either require technical expertise or come with expensive costs and limits. To address this gap this project proposes the design and implementation of an open-source platform for building domain specific conversational AI agents that exposes all its functionality via an Application Programming Interface (API), and aims to make the process of creating, customizing and using agents as simple as possible even for non-developers, while also being easy to extend and add-on to by developers.

1.2 Importance of the Project

Conversational AI is being applied in many domains such as customer service, education, healthcare and business. However, most current frameworks to build AI agents require technical expertise or need high costs, which limits the adoption of conversational AI agents by many organizations. This project aims to lower both the technical and financial barriers by introducing an open-source platform that enables enterprises, small organizations, and individuals to adopt AI capabilities with minimal costs and expertise. Additionally, the platform will serve as a foundation for developers.

1.3 Problem Statement

Despite the advancements in conversational AI, several gaps exist that keep limiting its adoption. First, many current frameworks prioritize graphical UIs instead of exposing APIs which limit their extensibility and integration capabilities. Second,

building conversational agents requires technical expertise such as “Langchain”, making them inaccessible to many potential users. Third, many frameworks are enterprise-focused and involve high licensing and subscription costs such as “RASA Studio”, making it difficult for smaller organizations and individuals to adopt this technology.

1.4 Objectives

- To Build a Complete RAG Pipeline that covers data ingestion, embedding and retrieval-based generation.
- To Architect and Implement a Modular System that exposes its functionality via an API. and to enable external function calling.
- To Develop a Web-Based Admin Panel for Agent Management.

1.5 Scope

In Scope:

- Develop an API using FastAPI as the core of the platform.
- Implement a modular architecture with clearly separate modules.
- Implement a knowledge ingestion pipeline that supports PDF documents.
- The ingestion pipeline will include text extraction, semantic chunking, vector embedding and storage.
- Implement a full RAG pipeline to answer questions based on ingested knowledge base.
- The agent will be able to use external tools (function calling).
- At least one external tool will be implemented and integrated (External API call).
- Build a functional admin panel to manage agents, allowing admins to upload knowledge files and manage tools.
- Provide auto-generated OpenAPI (Swagger) documentation.

Out of Scope:

- No AI training or fine-tuning. This project will use third party LLMs.
- The knowledge ingestion will be limited to PDFs. Support for other formats

will be considered future works (.docx, .txt, etc...).

1.6 Methodology

The platform will be architected as independent modules that communicate internally. The development methodology will be module-based, each module will be individually designed, developed, tested and documented before integrated.

Initially the core API and Retrieval-Augmented Generation (RAG) pipeline will be built, then an admin panel that consumes the API will be developed, followed by a final integration testing phase.

Core modules are:

- API Module.
- Knowledge Ingestion Module.
- Agent Core Module.
- Tools Module.

To validate the platform's efficacy, we will do a case study evaluation that involves creating multiple agents under a single organization, allowing for an assessment of the platform capabilities.

CHAPTER TWO: THEORETICAL FRAMEWORK AND LITERATURE REVIEW

2.1 Introduction

This chapter presents the theoretical foundations and related work for this project. It covers the main concepts used in the system. Then, it reviews existing systems and frameworks and identifies their limitations.

2.2 Theoretical Framework

This section explains the main concepts that the system is built on.

2.2.1 Large Language Models (LLMs)

Large Language Models (LLMs) are neural language models that are trained on large amounts of data and are able to understand and generate human-like text by predicting the next token in a text sequence. Being trained on large amounts of data allows them to serve as general-purpose systems as demonstrated in GPT3 (Brown et al. 2020).

Most modern LLMs are built on top of Transformers (Vaswani et al. 2017), which is a neural network architecture that works by processing all tokens in a sequence simultaneously instead of sequentially like Recurrent Neural Networks (RNNs).

However, LLMs have several limitations, one of them is hallucination (Ziwei Ji et al. 2022) in which LLMs generate incorrect information. Another one is knowledge cut-off which makes the model unaware of data made after training.

2.2.2 Retrieval-Augmented Generation (RAG)

To address hallucinations and knowledge limitations, Retrieval-Augmented Generation (RAG) (Lewis et al. 2020) has appeared as a solution. RAG works by retrieving relevant information from an external knowledge base and providing it to LLMs as context to generate a response. This allows LLMs to access up-to-date and domain-specific information and generate more verifiable and accurate responses.

2.2.3 Embeddings, Vector Databases, and Semantic Search

The RAG pipeline depends on embeddings, vector databases and semantic search to ingest, store, and retrieve information.

2.2.3.1 Embeddings

Embeddings are real numbers vectors that represent the semantic meaning of text. Embedding models convert text chunks into high dimensional vectors of hundreds or thousands of dimensions. Where words or text sequences with similar semantic meanings have closer numerical representation in the vector space. For example, OpenAI's text-embedding-3-small converts text chunks into 1536 dimensions vectors.

2.2.3.2 Vector Databases

A Vector database is a special type of databases that is designed to efficiently store and retrieve vector embeddings, unlike traditional databases that rely on exact matching, vector databases use nearest neighbor techniques (eg. Approximate Nearest Neighbor) and semantic similarity metrics to find information that is semantically similar to the query.

2.2.3.3 Semantic Search

Semantic search is a technique that retrieves information based on the meaning of the query instead of exact keyword matching. It works by calculating similarity between an embedded user query and stored vectors in a vector database using metrics such as cosine similarity, which measures the angle between two vectors in the vector space. The higher the cosine similarity score the more semantically relevant is the information to the query.

Together these three components form the RAG pipeline enabling the system to ingest, store and retrieve information to provide the LLM with factual data to use as a context for generating responses.

2.2.4 Tool Use

Tool use (Schick et al., 2023), also known as function calling, is the ability of LLMs to interact and use external systems such as calculators, APIs, search engines, and databases. It works by providing the LLM with a list of available tools with their descriptions and parameters. Based on the user query, the LLM decides which tool to invoke. The tool is then executed externally and the result is returned to the LLM to be used as context for the final response. Tool use enables LLMs to overcome knowledge and actions limitations by allowing them to interact with external systems.

2.2.5 Multi-tenancy Architecture

Multi-tenancy is a software architecture where a single instance of a system serves multiple tenants. In this work it allows multiple tenants to use the same LLM and RAG pipeline while keeping each tenant’s data completely isolated.

2.3 Literature Review

This section reviews related existing frameworks and systems and discusses their strengths and limitations.

2.3.1 Agents: An Open-source Framework for Autonomous Language Agents

The AGENTS framework (Zhou et al. 2023) introduces an open-source framework to make customizing, tuning, and deploying simpler while also remaining extensible by other developers. It supports long-short term memory, tool usage, web navigation, and multi-agent communication. AGENTS provides a novel paradigm to build controllable agents via a symbolic plan named as “Standard Operating Procedures” or SOPs. A SOP is a graph of state with their transition rules to provide controllability over the agent. The framework provides major features and introduces a novel controllability mechanism. However, while lowering the barrier for customizing agents. It still requires users to write long JSON configs to define SOPs and manage agents.

2.3.2 Design and Implementation of an AI-Based Chatbot Framework with Retrieval-Augmented Generation and Integrated Recommender System for Interactive User Support

The authors (Reddy et al. 2025) introduce a modular AI chatbot framework that integrates Retrieval-Augmented Generation (RAG) with a recommender system to offer context-aware support. The framework uses LangChain for orchestration, Ollama for running language models, FAISS for vector-based semantic search, and a custom recommender module. The framework's main strengths are its recommender system that suggests queries which enhances user experience, and its support for local deployment using Ollama. However, the architecture is designed as a single-instance architecture which limits multi-tenant deployment, and building a chatbot using it requires technical expertise.

2.3.3 FACTS About Building Retrieval Augmented Generation-based Chatbots

The authors (Akkiraju et al. 2024) present a framework for building effective enterprise-grade Retrieval Augmented Generation (RAG)-based chatbots based on their hands-on experience in developing three production chatbots at NVIDIA. FACTS is an acronym for five dimensions that RAG chatbots must get right. F- Freshness, A - Architecture, C - Cost Economic, T - Testing, and S - Security. The authors also present fifteen control points of RAG pipelines from data ingestion and chunking to answer generation. This paper provides a practical reference for designing and building efficient and secure chatbot systems.

2.3.4 Rasa Pro Framework

Rasa Pro (Rasa. 2019) is a framework for building scalable conversational AI agents. It uses CALM (Conversational AI with Language Models) (Bocklisch et al. 2024), which is a dialogue system that combines language models flexibility with predefined logic. It uses language models to interpret a user message and suggest next

steps in the conversation. The framework follows a flow-based dialogue management approach. Flows are predefined business processes that are broken into steps to guide the conversation, making them ideal for structured tasks with clear goals. However, manually defining these flows requires significant configuration effort, which may limit accessibility for non-technical users.

2.3.5 Bot Framework SDK

The Bot Framework SDK (Microsoft, 2016) is a collection of libraries, tools, and services that enable building, testing, deploying, and managing intelligent bots. It allows developers to build chatbots using multiple programming languages like C#, Python, Java, JavaScript. It can connect bots to multiple channels such as Slack, Teams and other applications. However, building and maintaining bots using the SDK requires technical expertise, which may limit accessibility for non-technical users.

2.3.6 Summary of Literature Review

Table 2.1 shows a summary of reviewed studies, their methods, results, and limitations.

Table 2.1 Summary of Literature Review

Study	Objectives	Methods	Results	Limitations
Agents: An Open-source Framework for Autonomous Language Agents	To build an open-source library and framework that make customizing, tuning, and deploying language agents simpler.	Python, Langchain, LLMs	Simplified the creation of single agent and multi agent systems.	The framework uses JSON files to define and manage agents, and is more developer-centric.
Design and Implementation of an	the design and implementation of a	NLP (NLTK,	The framework demonstrates	It is a single-instance system, designed to

AI-Based Chatbot Framework with Retrieval Augmented Generation and Integrated Recommender System for Interactive User Support	modular chatbot framework that combines conversational AI with retrieval-augmented generation (RAG) and an embedded recommender system to enhance user interaction.	SpaCy), RAG, Langchain, OLLama, FAISS, TF-IDF,	significant improvements in both response accuracy and user experience when compared to traditional and existing models	build one chatbot, and requires technical expertise.
FACTS About Building Retrieval Augmented Generation-based Chatbots	To design a framework for building effective RAG-based chatbots based on their experience of building three chatbots at NVIDIA	RAG	It offers a holistic perspective on the essential factors and practical solutions for building secure and efficient enterprise-grade chatbots.	It offers a set of principles and best practices, not a reusable tool.

Rasa Pro Framework	A framework for building scalable, high-trust conversational AI assistants.	NLU, LLMs, Python	It offers a framework for building AI assistants using (CALM) approach that utilizes LLMs for interpreting user input ensuring agents following the predefined flows.	It requires the manual creation of YAML files to define all conversations flows. The architecture overall is optimized for task-oriented dialogue with RAG as an add-on. The RASA Studio (no-code tool) has high license fees.
Bot Framework SDK	Tools for building and managing smart bots.	Language Understanding (LUIS), C#, Node.js, Dialogue Management,	a collection of libraries, tools, and services that let you build, test, deploy, and manage intelligent bots.	It requires technical expertise, has a steep learning curve, follows a procedural paradigm where the developers define the whole conversation flow, highly optimized for Azure cloud, RAG is not a feature and has to be integrated manually.

2.4 Summary

This chapter reviewed the theoretical concepts and related work for this project. All the reviewed systems and frameworks offer strong capabilities, yet they share common limitations such as requiring technical expertise, lacking multi-tenancy support

or not having RAG as a built-in feature. The proposed project aims to address these limitations by building a multi-tenant platform that makes building RAG-powered AI agents simpler.

CHAPTER THREE: METHODOLOGY

3.1 Introduction

This chapter describes the software development methodology and the system design and analysis. It begins by showing the chosen development model, followed by the system requirements. Then, it presents the system architecture and the system design. And finally, it describes the data design and the evaluation plan of the project.

3.2 Development Methodology

The platform was built following a bottom-up incremental approach. Development started from the core layer, which includes three modules: the knowledge processing module, the tool use module, and the chat module. Then the API layer was developed which consists of several domains: agents, auth, conversations, organizations, and users. Finally, the admin panel layer was built on top of the API. Each component went through its own cycle of design, development and manual testing before being integrated.

This approach was chosen because each upper layer completely depends on the one below it. The admin panel depends on the API and the API depends on the core layer.

3.3 Requirements Analysis

3.3.1 Functional Requirements

- The users shall be able to register for and login to the system.
- An authenticated user shall be able to create an organization or join one with an invitation.
- An organization owner or admin shall be able to invite other users to the organization.
- An organization owner or admin shall be able to assign roles and permissions to users in the organization.
- A user with the needed permissions shall be able to manage agents, knowledge bases, and tools.
- The system shall be able to ingest uploaded pdf documents into a vector store.

- The agent shall answer end-users' queries using only context from its knowledge base and its tools.

3.3.2 Non-Functional Requirements

- Multi-tenancy Isolation: All data shall be isolated by organization. A user shall never be able to access data belonging to an organization they are not a member of.
- Usability: Users shall be able to perform all workflows of the system through the admin panel.
- Maintainability: The codebase shall be organized into three separate layers: the core, API, and admin panel. Where each layer consists of smaller modules ensuring separation of concerns.
- Security: All API endpoints shall require authentication except registration and login.

3.3.3 Use Case Diagram

Figure 3.1 outlines the main functionalities and actors of the platform.



Figure 3.1 Use Case Diagram

3.4 System Architecture

The platform follows a layered architecture with three layers: UI, API, and Core layer. The three layers depend on a set of infrastructure and external services that include PostgreSQL, Qdrant, an LLM API, and third-party APIs, As Illustrated in Figure 3.2.

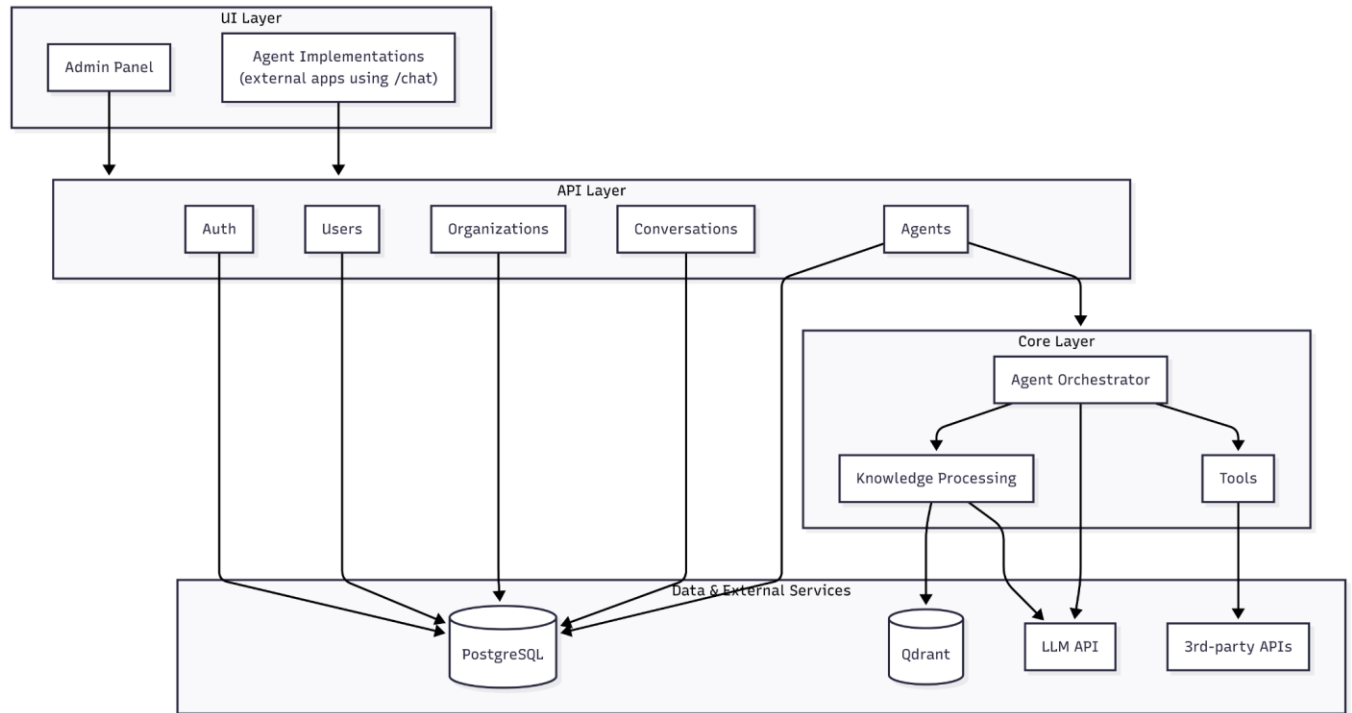


Figure 3.2 System Architecture

3.4.1 UI Layer

The UI layer contains two types of clients: the admin panel which provides a management interface for configuring agents, users and organizations. An external agent implementation that integrates with the system to build conversational applications such as a website widget or a mobile app.

3.4.2 API Layer

The API layer exposes the system's functionality through a RESTful API. The layer is composed of five modules: auth, users, organizations, conversations, agents.

3.4.3 Core Layer

The core layer contains the main logic for the agents. It is composed of three modules: the agent orchestrator which runs the chat loop, the knowledge processing module which is responsible for ingesting and retrieving data, and the tools modules which convert and execute tools.

3.4.4 Data and External Systems

This tier covers the infrastructure and external dependencies which the platform uses. PostgreSQL is used as the primary store for relational data. Qdrant is used for vector storage and searching. The LLM API provides the large language model needed for generating responses. And the third-party APIs extend the system capabilities using tool integrations.

3.5 System Design

This section describes the system design through the main workflows of the system using class, activity, and sequence diagrams.

3.5.1 Class Diagram

The class diagrams show the main classes of the system along with their attributes.

Figure 3.3 presents organizations and users, while the Figure 3.4 presents agents.

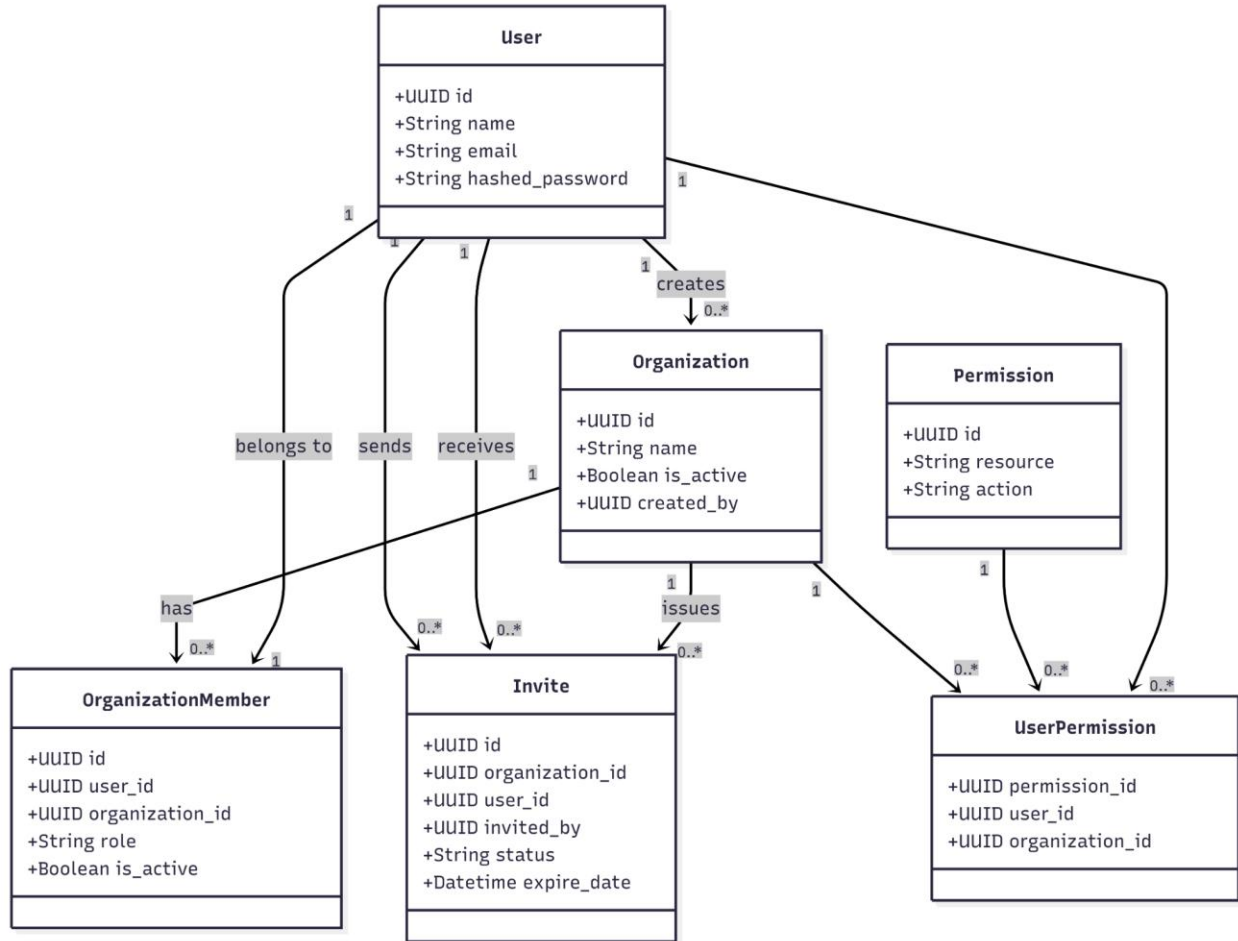


Figure 3.3 Organizations and Users Class Diagram

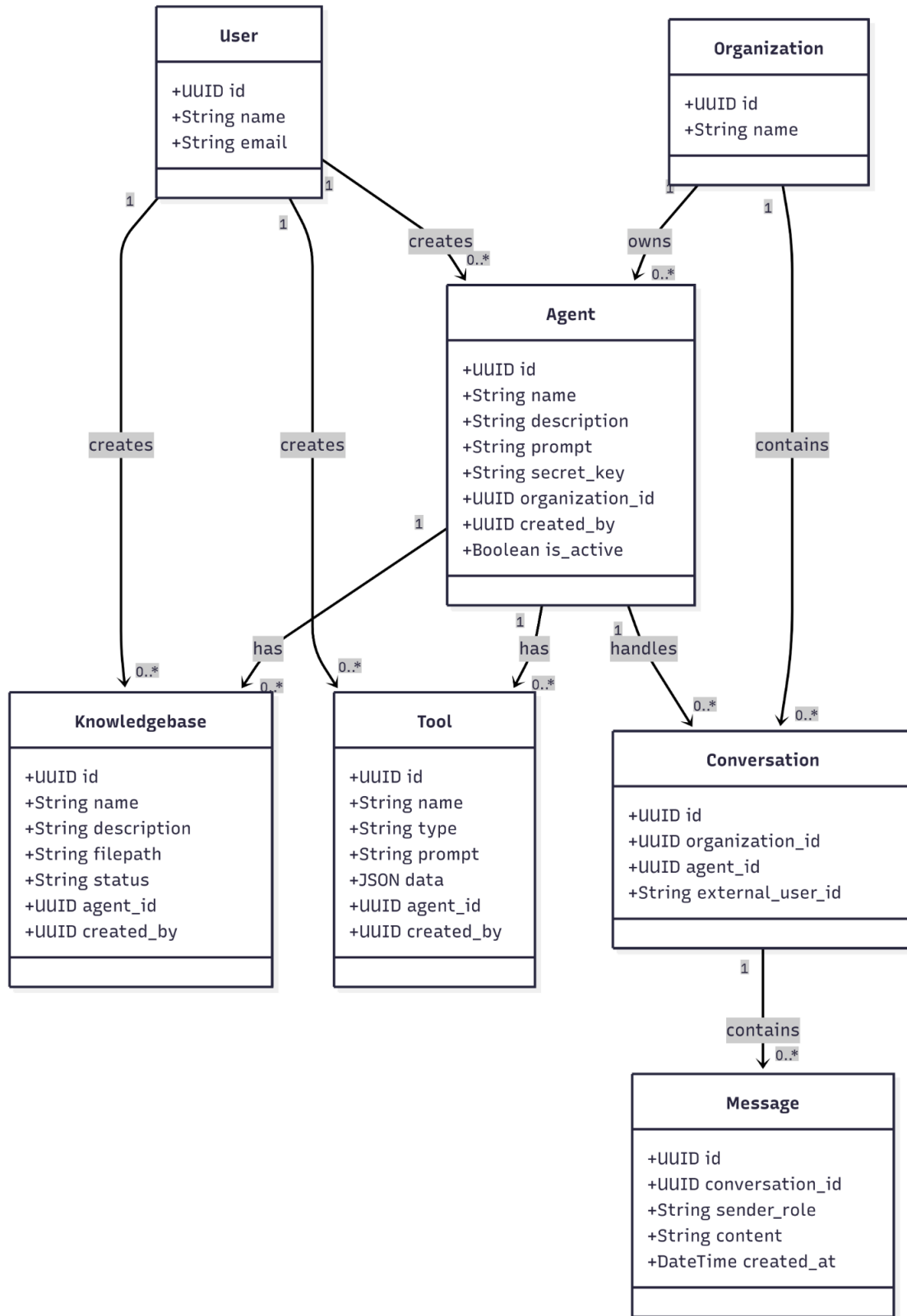


Figure 3.4 Agents Class Diagram

3.5.2 Knowledge Ingestion

The knowledge ingestion process begins with loading a PDF file, then splitting its text into chunks. Each chunk is then embedded and the resulting vectors are stored in a vector database along with its metadata that links them to an agent and an organization, as illustrated in Figure 3.5.

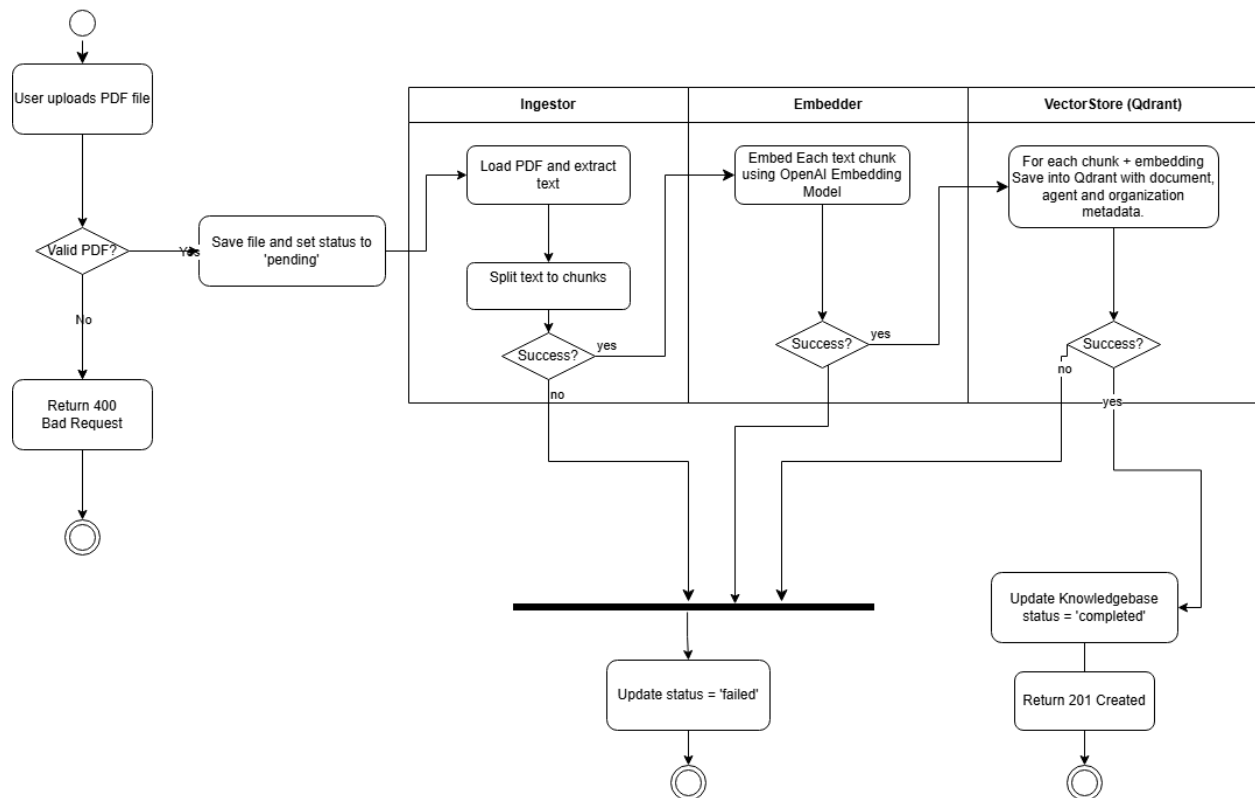


Figure 3.5 Knowledge Ingestion Activity Diagram

3.5.3 Chat

The chat process begins with authenticating the request, loading the conversation history and agent's tools, decomposing the query into subqueries, retrieving relevant context from the vector database for each query, preparing the prompt and sending the request to the large language model, handling tool calling if required, and finally saving the messages and returning the response, as shown in Figure 3.6.

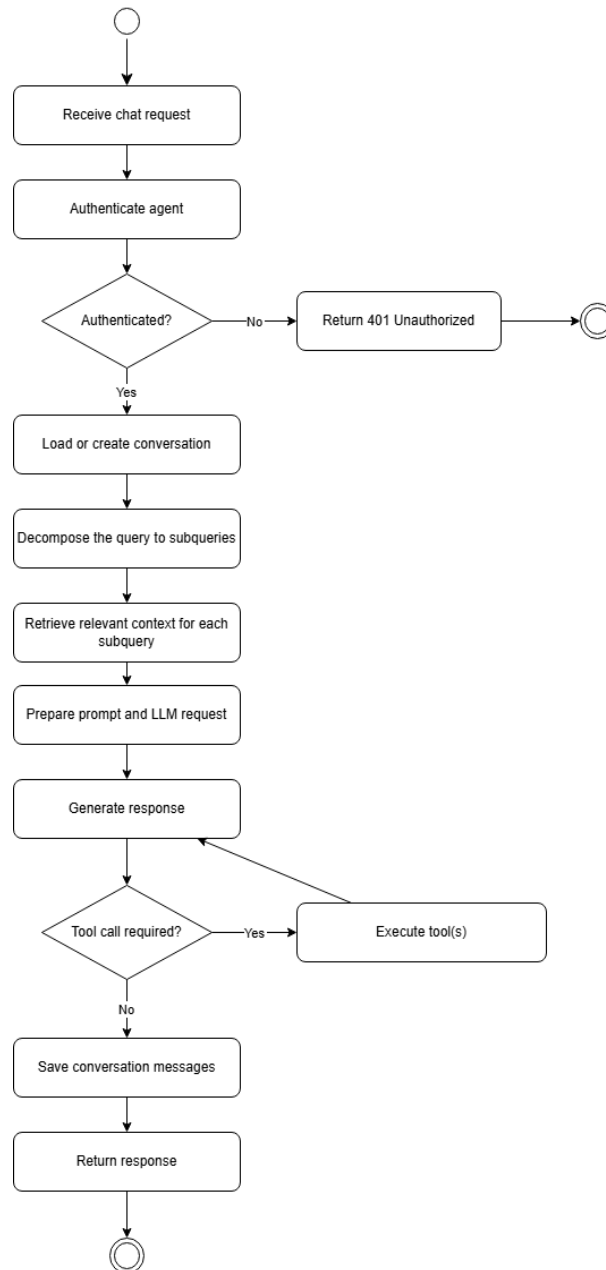


Figure 3.6 Chat Activity Diagram

The flow and order of interactions between system components during a chat request is shown in Figure 3.7.

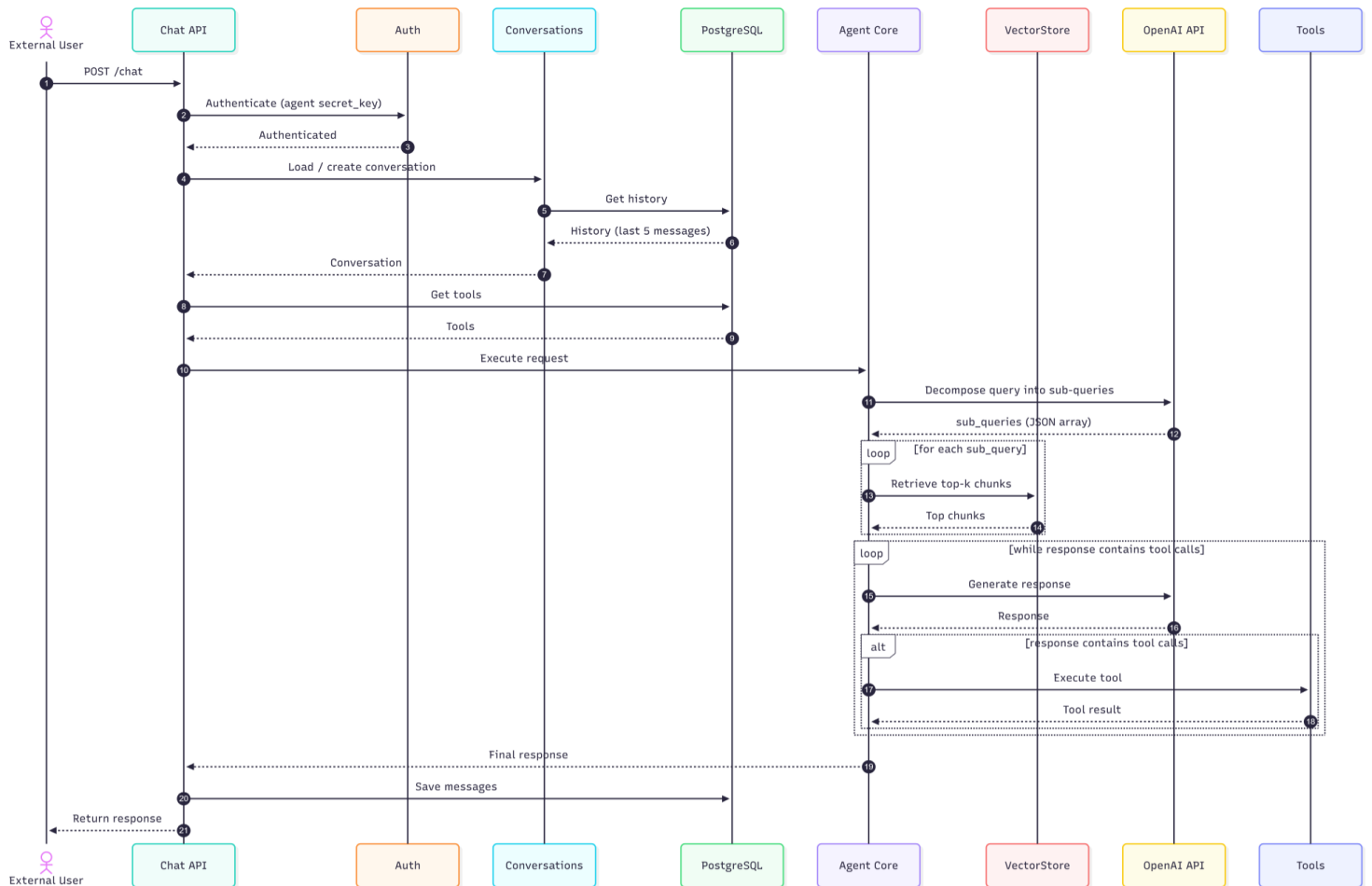


Figure 3.7 Chat Sequence Diagram

3.6 Database Design

The system uses two databases: a relational database for storing structured data such as users, organizations, agents, and conversations, and a vector database for storing document embeddings used in knowledge retrieval. Each point in the vector database contains the embedding along with metadata that includes the organization, agent, and document IDs which are used for filtering. The relational database schema is shown in Figure 3.8.

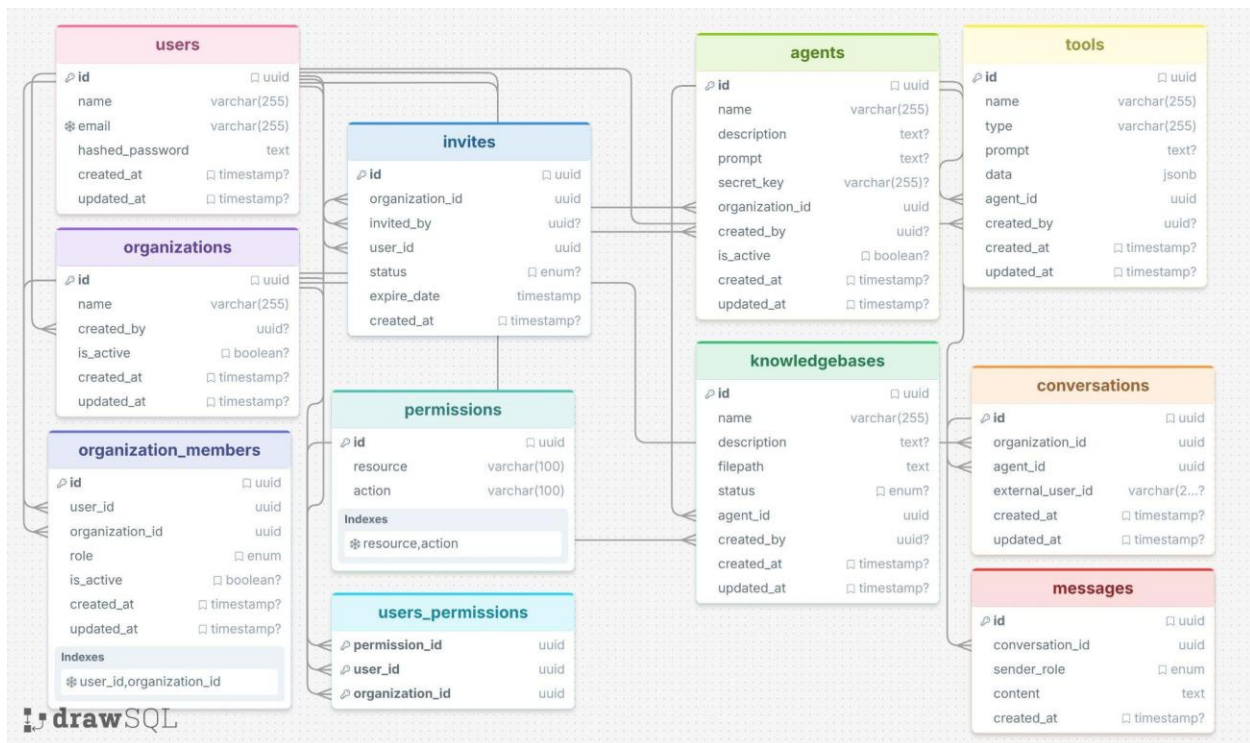


Figure 3.8 ERD Diagram

3.7 Evaluation Plan

The platform will be evaluated from three different aspects. Functional evaluation will verify that the system meets the requirements. Usability evaluation will be assessed using the System Usability Scale (SUS), where users will use the admin panel and complete a questionnaire. AI Evaluation will be evaluated using Automated Evaluation of

Retrieval Augmented Generation (RAGAS) framework (Es et al, 2025). to evaluate the quality of the RAG pipeline, alongside manual tool calling tests to verify that tool selection, arguments passing, and result integration are being handled correctly.

3.8 Summary

This chapter described the development methodology, system analysis, architecture, and design of the platform.

CHAPTER FOUR: IMPLEMENTATION

4.1 Introduction

This chapter presents how the platform was implemented. It begins by listing the tools and technologies used, followed by the implementation details of each layer of the platform.

4.2 Tools and Technologies

This section outlines the tools and technologies used to build the platform, covering backend, frontend, infrastructure, and AI models.

4.2.1 Backend

The backend is implemented using Python and uses the technologies listed in Table 4.1.

Table 4.1 Backend Technologies

Technology	Use
FastAPI	The main web framework used for building the API and generating OpenAPI documentation.
SQLAlchemy	ORM used for querying the relational database.
Pydantic	Data validation library used for creating request and response schemas for validation.
OpenAI Python Library	A library for accessing OpenAI REST API used for communicating with AI models.
Qdrant Client	Client library and SDK for Qdrant vector database used for storing and retrieving document embeddings.
PyMuPDF	A python library used for text extraction.
Langchain Text Splitter	A library for text splitting Used for chunking extracted text.
Bcrypt	An encryption library used for hashing passwords.

4.2.2 Frontend

The frontend is implemented using Typescript and uses the technologies listed in Table 4.2.

Table 4.2 Frontend Technologies

Technology	Use
React	A component-based UI library used for building the admin panel interface.
Vite	A frontend build tool and a dev server used for deploying the admin panel.

4.2.3 Infrastructure and Tools

The platform uses the databases, tools and infrastructure components listed in Table 4.3.

Table 4.3 Infrastructure and Tools

Technology	Use
PostgreSQL	The primary relational database used for storing structured data.
Qdrant	A Vector database used for storing document embeddings.
Docker	A Containerization platform used for running Qdrant.
Git	A version control system used for managing the codebase.
Uvicorn	A web server used for serving the API.

4.2.4 AI Models

The platform uses two AI models for its main functionality, listed in Table 4.4.

Table 4.4 AI Models

Model	Use
gpt-5-mini	A large language model used for agent response generation and query decomposition.
text-embedding-3-large	An embedding model used for generation vector embeddings.

4.3 Core Layer

The core layer is responsible for all agent behaviour. It manages how an agent processes users' messages, retrieves knowledge, and interacts with external tools. The layer is divided into three modules.

4.3.1 Knowledge Processor

The knowledge processing module is responsible for the RAG pipeline, which begins with ingesting PDF documents, splitting text into chunks using LangChain's RecursiveCharacterTextSplitter with a chunk size of 1000 characters and an overlap of 200 characters, converting them into vector embeddings, and storing them in the vector database for semantic retrieval. The module is divided into five classes:

- **Ingestor:** handles PDF document loading and text chunking.
- **Embedder:** converts text to vector representations using the embedding model.
- **VectorStore:** manages the vector database operations including adding, searching, and deleting embeddings.
- **Retriever:** retrieves relevant chunks by embedding a user query using the Embedder and searching using the VectorStore.
- **Processor:** coordinates processing a pdf file by using the Ingestor, Embedder, and VectorStore sequentially.

4.3.2 Tool Use

The tool use module enables agents to interact with external services by calling external functions. It consists of two classes:

- **Converter:** transforms tools definition into the format required by the LLM provider API.
- **Executor:** handles functions that the agents can execute including calling an external API.

4.3.3 Chat

The chat module is responsible for receiving a user message and generating a response by combining prompts, retrieved knowledge, and tools. It consists of two classes.

- **LLM:** Initializes and interacts with the LLM provider (OpenAI) SDK to generate responses.
- **Agent:** It is the main orchestrator that implements the agent execution pipeline. It begins by decomposing the query into subqueries, then retrieving relevant knowledge for each subquery using the Retriever. If there are no relevant chunks or tools defined it returns a default fallback message. Otherwise, it assembles the system prompt, converts tools to the OpenAI format, and calls the LLM. If the model returns a tool call, the agent enters a loop that executes the function call, appends the output, and calls the LLM again. The loop will continue until there are no more tool calls or it reached the iteration limit, as illustrated in Figure 4.1.

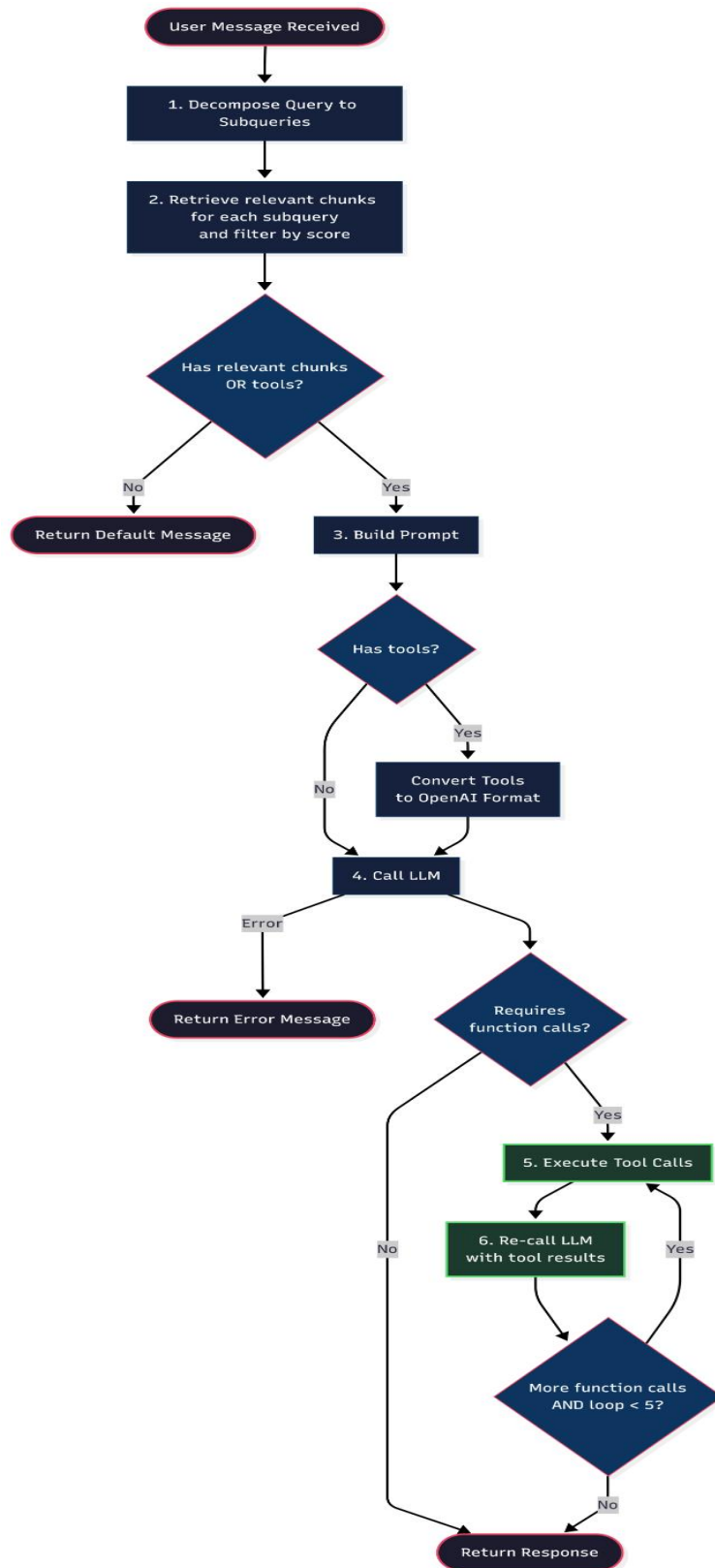


Figure 4.1 Agent Execution Pipeline

4.4 API Layer

The API layer exposes the platform's functionality through a set of RESTful endpoints.

4.4.1 Structure

The API is organized into five domains:

- **Auth:** handles user registration and login.
- **Organizations:** handles organization CRUD operations, member management, and invitations.
- **Users:** allows users to view, accept and decline organization invitations.
- **Agents:** handles CRUD operations for AI agents, including their knowledge bases and tools.
- **Conversations:** manages conversations history and handles chat interactions.

Each domain follows a four files structure:

- **models.py:** defines SQLAlchemy ORM models.
- **schemas.py:** defines Pydantic schemas for request and response validation.
- **service.py:** contains the business logic and performs database operations.
- **routes.py:** defines the API endpoints and maps them to service functions.

4.4.2 Authentication and Authorization

The platform implements two authentication mechanisms and a role and permission-based authorization model to secure the endpoints.

User authentication is used in all operations of the platform. When a user logs in successfully, a JWT token is issued containing the user id. Following requests must include this token in the authorization bearer header.

Agent authentication is used for the chat endpoint, enabling external applications to interact with the agent. When an agent is created a secret key is configured and stored as a hash. Following chat requests must include this key in the request body.

Authorization follows a role and permission-based model where the system has three roles:

- **Owner:** can perform all operations, including deleting the organization.
- **Admin:** manages all resources and members, except other admins and the owner.
- **User:** can only perform operations for which they have been granted permissions.

Permissions are assigned by admins and the owner for each user individually.

4.5 Admin Panel

The admin panel is a single page application that provides a management interface for the platform. It communicates directly with the API, with each interface action mapping to an API operation. Screenshots are available in Appendix A.

4.6 Database

The platform uses two databases to handle different types of data. Relational data is managed by PostgreSQL. Vector data is managed by Qdrant, which is configured as a single collection that stores document chunks embeddings as 3072 dimensions vectors.

4.7 Summary

This chapter presented the implementation of the platform, covering the tools and technologies used, the development of the core, API, and admin panel layers, and the databases the system uses.

CHAPTER FIVE: RESULTS AND RECOMMENDATIONS

5.1 Introduction

This chapter presents the results of the system evaluation and recommendations for future improvements. The evaluation was done on three different aspects: a functional evaluation that verifies each feature behaves as expected, a usability evaluation that collects feedback from users on the admin panel experience, and a core evaluation that measures the quality of the RAG and tool use implementation.

5.2 Evaluation and Results

The system was evaluated using three different methods, each targeting a different aspect of the system: functional tests to verify correctness, a user survey to assess usability, and AI and statistical metrics to measure the quality of the core AI components.

5.2.1 Functional Evaluation

The functional evaluation verifies that each feature of the system behaves as expected. Each feature was tested manually using the admin panel through black-box testing, which works by testing system output against expected results. The tested features include user registration and login, organization and member management, agent creation and configuration, knowledge base ingestion, and chatting with an agent, with the results listed in Table 5.1.

Table 5.1 Black-box Testing.

Function	Input	Expected Result	Actual Result	Status
Login	Valid email and password	User is redirected to dashboard	User successfully accessed dashboard	Pass
Register	Valid name, email, and password	New account is created successfully	Account created successfully	Pass
Create Agent	Valid agent name, description, and prompt	Agent is saved in the system	Agent created successfully	Pass
View Agent	Select an existing agent	Agent information is displayed	Agent information displayed correctly	Pass
Update Agent	Modified agent name or prompt	Updated information is saved	Changes saved successfully	Pass
Delete Agent	Select an existing agent and confirm deletion	Agent is removed from the system	Agent removed successfully and not	Pass
Upload Knowledge Source	Valid PDF document and name	Document is processed and indexed	Document uploaded and indexed successfully	Pass
View Knowledge Sources	Navigate to knowledge base section	Uploaded documents are displayed	Documents displayed correctly	Pass
Delete Knowledge Source	Select a document and confirm deletion	Document is removed from the knowledge base	Document deleted successfully	Pass

Create Tool	Valid tool name, type, and configuration	Tool is added to the system	Tool created successfully	Pass
View Tools	Navigate to tools section	Tool information is displayed	Tool information displayed correctly	Pass
Update Tool	Modified tool settings	Changes are saved	Tool updated successfully	Pass
Delete Tool	Select a tool and confirm deletion	Tool is removed from the system	Tool deleted successfully	Pass
View Conversations	Navigate to conversations section	Previous conversations are displayed	Conversations displayed correctly	Pass
Chat with Agent	Message sent to an agent	Agent generates a response	Response generated successfully	Pass
Create Organization	Organization name	Organization is created successfully	Organization created successfully	Pass
View Organization	Navigate to organization settings	Organization information is displayed	Information displayed correctly	Pass
Delete Organization	Owner delete organization	Organization is removed from the system	Organization deleted successfully	Pass
Leave Organization	Member leaves an organization	User is removed from organization membership	User left organization successfully	Pass

Transfer Ownership	Owner selects a member and transfers ownership	Ownership is transferred to another member	Ownership transferred successfully	Pass
Remove Member	Owner or Admin selects a member and remove	Selected member is removed	Member removed successfully	Pass
Invite User	User to invite ID	Invitation is sent to the user	Invitation sent successfully	Pass
Accept Invitation	Accept invitation	User becomes a member of the organization	User joined organization successfully	Pass
Decline Invitation	Decline invitation	Invitation is rejected	Invitation declined successfully	Pass
Assign Permissions	Assign permissions to a selected user	Permissions are updated	Permissions assigned successfully	Pass
Change Role	Change the role of a selected member	Member role is updated	Role changed successfully	Pass

All tested features behaved as expected. Authenticated users could only access resources they have permission for, uploaded PDF documents were ingested into the knowledge base, and agents responded using retrieved context or returned to the default fallback message when no relevant information was available.

5.2.2 Usability Evaluation

The usability evaluation assesses how easy and satisfying is the admin panel to use, covering the main workflows of the system.

The evaluation was done on 10 participants. Each was asked to create an account, login, create an organization, create an agent, upload a knowledge base, and ask the agent

a few questions. After completing the tasks each participant filled a System Usability Scale (SUS) questionnaire, which contains 10 statements rated on a five-point scale. The responses were then converted to a SUS score using the standard SUS formula.

The average SUS score across the participants was 83.5 which indicates that the participants found the admin panel easy to use. Table 5.2 lists the average rating per statement.

Table 5.2 SUS Results

Statement	Average Rating (1-5)
I think that I would like to use this system frequently.	4
I found the system unnecessarily complex.	1.5
I thought the system was easy to use.	4.3
I think that I would need the support of a technical person to be able to use this system.	1.7
I found the various functions in this system were well integrated.	4.5
I thought there was too much inconsistency in this system.	1.6
I would imagine that most people would learn to use this system very quickly.	4.7
I found the system very cumbersome to use.	1.6
I felt very confident using the system.	4
I needed to learn a lot of things before I could get going with this system.	1.7

5.2.3 Core AI Evaluation

The core AI evaluation measures the quality of the two components of an agent: the retrieval-augmented generation (RAG) pipeline and the tool use functionality. The RAG evaluation assesses the agent retrieval and generation capabilities, while the tool

use evaluation assesses whether the agent selects the correct tool with the correct parameters and uses the result to generate a correct answer.

5.2.3.1 Retrieval-Augmented Generation Evaluation

The evaluation was performed using RAGAS Which is an open-source framework that measures the quality of RAG systems across four metrics: context recall and context precision for evaluating the quality of the retrieval step, and faithfulness and answer relevancy for the generation step.

- **Context Precision** evaluates whether retrieved contexts are useful for answering a question.
- **Context Recall** measures how many of the relevant documents were successfully retrieved.
- **Faithfulness** measures how factually consistent a response is with the retrieved context.
- **Answer Relevancy** measures how relevant a response is to the user input.

The evaluation was conducted on the Sudan University of Science and Technology academic guidebook. A test set of 31 questions was generated by an LLM and reviewed manually. The questions were grouped into four categories: simple factual (12 questions), definition (7 questions), multi-chunk (7 questions), and out of scope questions (5 questions) whose answers are not in the document. For each question a reference answer was written to enable the measurement of context recall. To determine the optimal retrieval configuration, the test was evaluated across different combinations of chunk sizes and minimum similarity score. The results in Table 5.3 contain the in-scope questions only since the metrics do not apply to out-of-scope questions.

Table 5.3 RAGAS Results by Configuration

Config\Metric	Context Precision	Context Recall	Faithfulness	Answer Relevancy
Chunks = 7 Min_score = 0.3	0.648	0.855	0.900	0.739
Chunks = 7 Min_score = 0.5	0.288	0.269	0.292	0.196
Chunks = 7 Min_score = 0.25	0.659	0.894	0.933	0.749
Chunks = 5 Min_score = 0.25	0.618	0.893	0.953	0.615
Chunks = 10 Min_score = 0.25	0.630	0.897	0.918	0.765

For the out-of-scope questions, the agent was evaluated on whether it declined to answer rather than hallucinating a fake response. The agent successfully refused all five out-of-scope questions, achieving a 100% refusal accuracy.

The results show that the minimum similarity score has the largest impact on the metrics. At a minimum score of 0.5, all metrics go down because many relevant chunks are filtered out leaving the agent with less context to generate a good response. As for the other configurations, retrieving fewer chunks leads to higher faithfulness but lower answer relevancy, while retrieving more chunks leads to higher recall and relevancy but lower precision.

The configuration with 7 chunks and a 0.25 minimum score offers the best overall balance, achieving the highest context precision (0.659), high context recall (0.894), faithfulness (0.933), and answer relevancy (0.749), and therefore was selected as the final configuration of the system.

The high faithfulness and the 100% refusal accuracy confirm the system's core requirement of answering only from the knowledge base and declining when no relevant

context is available. The high context recall indicates that the retriever is fetching the needed information, while the lower context precision means that some of the retrieved chunks are less relevant.

5.2.3.2 Tool-use Evaluation

The tool use evaluation was done following an approach adapted from API-Bank (Li et al., 2023), a benchmark for evaluating tool-augmented LLMs based on the correctness of API calls and the quality of LLM responses. Adapting this framework, the evaluation process assesses three aspects: tool selection, which is whether the agent chooses the correct tool, parameter accuracy, which is whether the call arguments match the expected values, and result integration, which is whether the agent uses the tool output correctly in the final response.

The evaluation used a set of six tools simulating a school academic system: get course info by course code, get student info by ID, get grades by ID, get gpa by ID, get schedule by ID and term, find student ID by name. Each tool has an endpoint that returns a fixed response. A test set of 17 cases of different difficulties was prepared by an LLM and reviewed manually, covering single tool calls, multi tool queries, multi turn cases that require sequential dependent calls where the output of a tool is the input of the next, and negative cases where no tool should be called. For each case the tools, parameters and expected behaviour were defined in advance and tested manually.

The agent achieved 88% tool selection accuracy, 100% parameter accuracy on correct tool calls, and 100% result integration accuracy across the 17 test cases. It handled multi turn dependent call chains successfully and declined to call a tool in out-of-scope questions. The two failures occurred when the agent called a tool it judged relevant for a question that no tool supported, but it then correctly reported that the information was not available.

The evaluation is limited by the set of tools and the size of the test set, so the result may not be accurate for larger and more complex tool sets.

5.3 Recommendations for Future Work

Several improvements can be made to extend and refine the platform's capabilities. In terms of knowledge ingestion, the current ingestion pipeline is limited to PDF files. Supporting additional formats such as .csv, .docx, and web URLs would make the platform more flexible. Additionally, adding Optical Character Recognition (OCR) support would allow the platform to process scanned documents that cannot be parsed through the standard text extraction process. As for knowledge retrieval, the RAGAS evaluation revealed that the context precision was the weakest metric, meaning that some of the retrieved chunks were not relevant to the query. Adding a re-ranking step using a cross-encoder model would likely improve both precision and overall response quality.

Regarding tool use, the current implementation supports only a single tool type. Adding additional tool types would expand the range of tasks an agent can perform. Additionally, adding support for the Model Context Protocol (MCP) would allow the platform to leverage the growing ecosystem of MCP-compatible tools, further expanding its capabilities.

REFERENCES

- Akkiraju, R., Xu, A., Bora, D., Yu, T., An, L., Seth, V., Shukla, A., Gundecha, P., Mehta, H., Jha, A., Raj, P., Balasubramanian, A., Maram, M., Muthusamy, G., Annepally, S. R., Knowles, S., Du, M., Burnett, N., Javiya, S., . . . Boitano, J. (2024). *FACTS about building retrieval augmented generation-based chatbots*. arXiv. <https://arxiv.org/abs/2407.07858>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., . . . Amodei, D. (2020). *Language models are few-shot learners*. arXiv. <https://arxiv.org/abs/2005.14165>
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated evaluation of retrieval augmented generation*. arXiv. <https://arxiv.org/abs/2309.15217>
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Chen, D., Dai, W., Chan, H. S., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38. <https://doi.org/10.1145/3571730>
- LangChain. (2022). *LangChain*. <https://www.langchain.com/>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv. <https://arxiv.org/abs/2005.11401>
- Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., & Li, Y. (2023). *API-Bank: A comprehensive benchmark for tool-augmented LLMs*. arXiv. <https://arxiv.org/abs/2304.08244>
- Microsoft. (2016). *Microsoft Bot Framework*. <https://learn.microsoft.com/en-us/azure/bot-service/bot-service-overview?view=azure-bot-service-4.0>
- Rasa. (2019). *Rasa Pro framework*. <https://rasa.com/docs/pro/intro/>
- Reddy, N. T. K. (2025). *Design and implementation of an AI-based chatbot framework with retrieval augmented generation and integrated recommender system for interactive user support*. SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5250507

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language models can teach themselves to use tools*. arXiv. <https://arxiv.org/abs/2302.04761>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. arXiv. <https://arxiv.org/abs/1706.03762>

Zhou, W., Jiang, Y. E., Li, L., Wu, J., Wang, T., Qiu, S., Zhang, J., Chen, J., Wu, R., Wang, S., Zhu, S., Chen, J., Zhang, W., Tang, X., Zhang, N., Chen, H., Cui, P., & Sachan, M. (2023). *Agents: An open-source framework for autonomous language agents*. arXiv. <https://arxiv.org/abs/2309.07870>

APPENDIX A: UI SCREENS

Figures A.1 to A.15 shows screenshots of the admin panel's main interfaces.

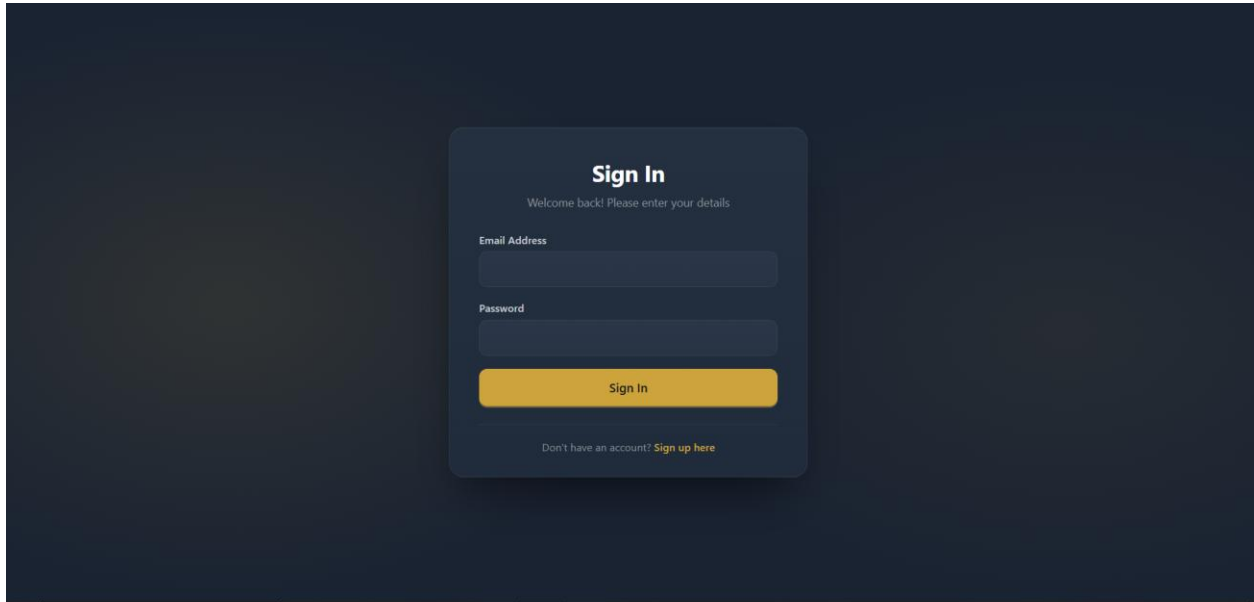
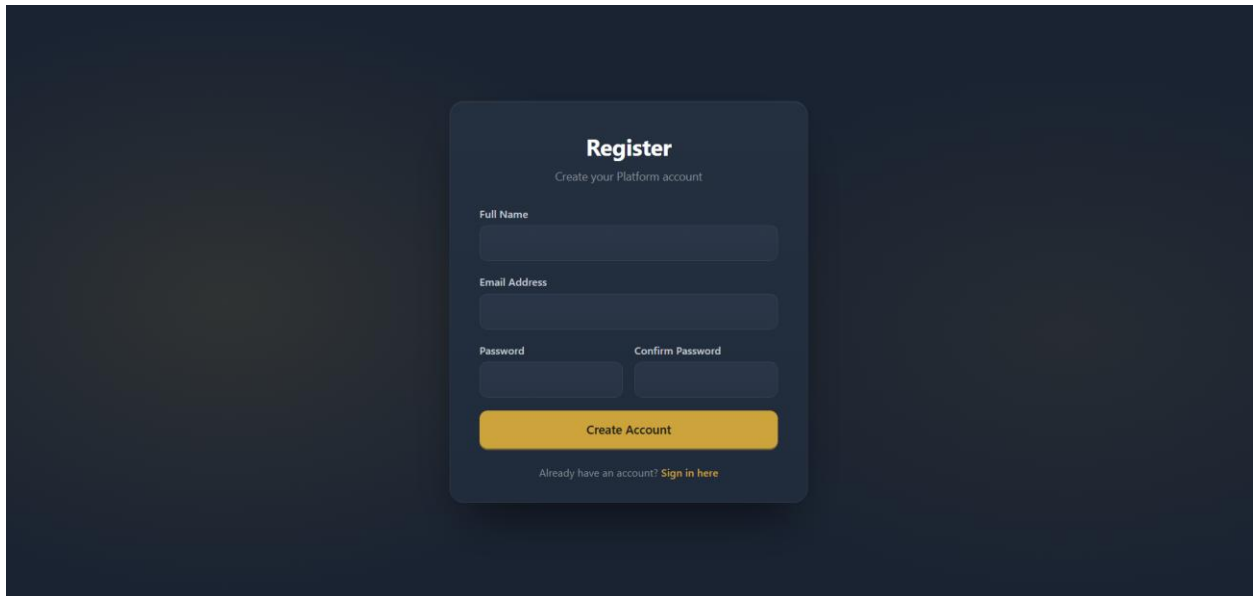


Figure A.1: Sign In Page



The registration page features a central form titled "Register" with the subtitle "Create your Platform account". The form includes input fields for "Full Name", "Email Address", "Password", and "Confirm Password". A yellow "Create Account" button is positioned below the password fields. At the bottom of the form, a link "Sign in here" is provided for users who already have an account.

Register
Create your Platform account

Full Name

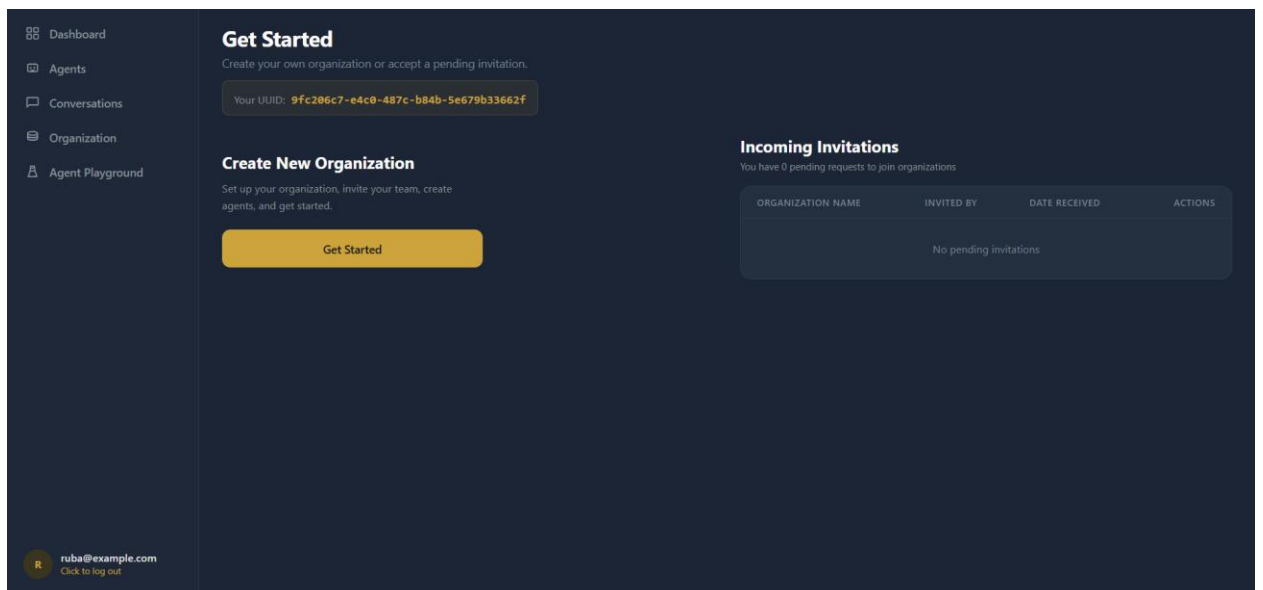
Email Address

Password Confirm Password

Create Account

Already have an account? [Sign in here](#)

Figure A.2: User Registration Page



The dashboard page for a user not in an organization includes a sidebar with navigation links: Dashboard, Agents, Conversations, Organization, and Agent Playground. The main content area is titled "Get Started" and contains a "Create New Organization" section with a "Get Started" button. A "Your UUID" box displays the value "9fc206c7-e4c0-487c-b84b-5e679b33662f". The "Incoming Invitations" section shows "You have 0 pending requests to join organizations" and a table with columns for Organization Name, Invited By, Date Received, and Actions. The footer shows the user's email "ruba@example.com" and a "Click to log out" link.

Get Started
Create your own organization or accept a pending invitation.

Your UUID: 9fc206c7-e4c0-487c-b84b-5e679b33662f

Create New Organization
Set up your organization, invite your team, create agents, and get started.

Get Started

Incoming Invitations
You have 0 pending requests to join organizations

ORGANIZATION NAME	INVITED BY	DATE RECEIVED	ACTIONS
No pending invitations			

ruba@example.com
Click to log out

Figure A.3: Dashboard Page (when you are not in an organization)



Figure A.4: Dashboard Page (when you are in an organization)

The 'Create New Organization' modal form includes the following fields and buttons:

- Organization Name: e.g. Acme Inc.
- Description (Optional): What does your team do?
- Buttons: Cancel, Create Organization

Figure A.5: Create New Organization Page

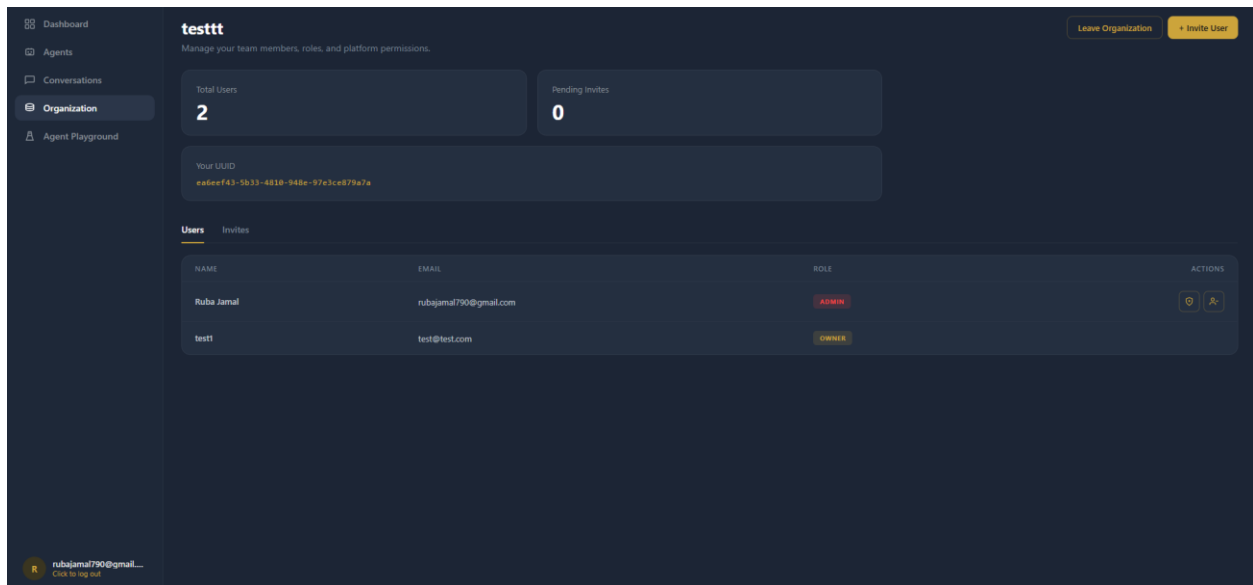


Figure A.6: Organization Configuration Page

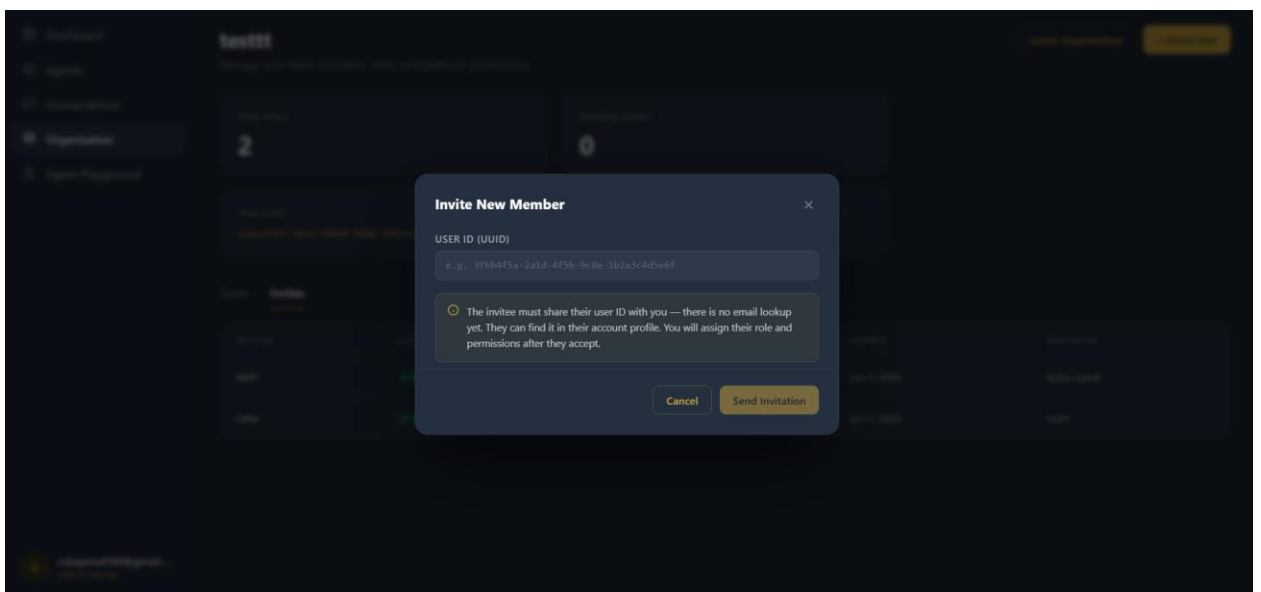


Figure A.7: Invite New Member Page

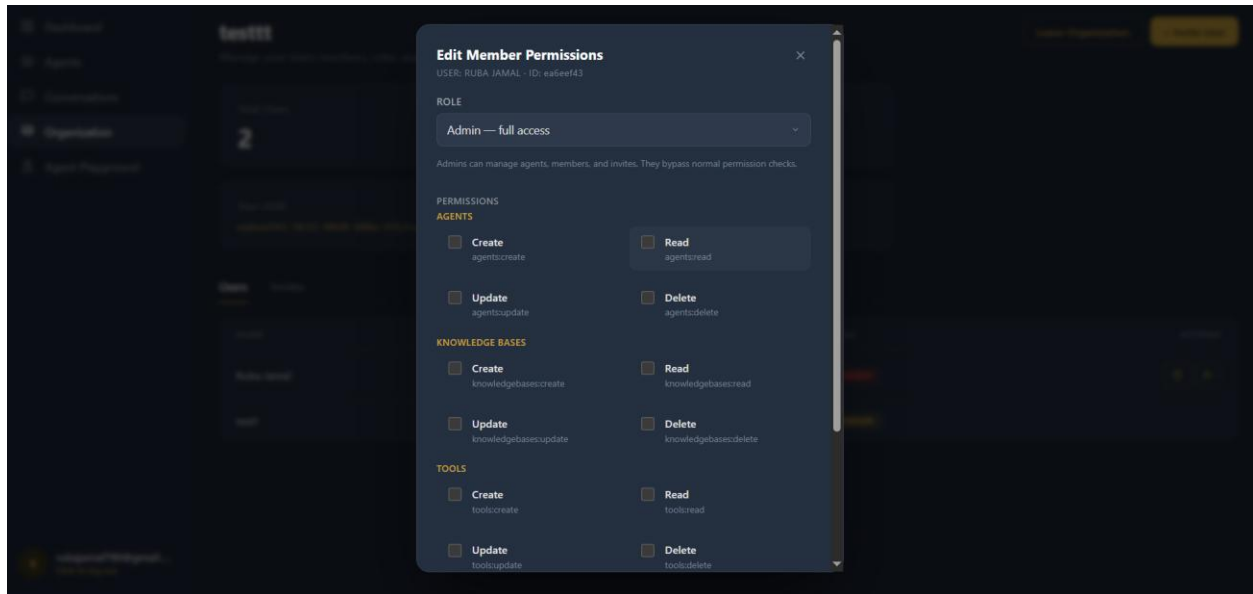


Figure A.8: Edit Member Permission Page

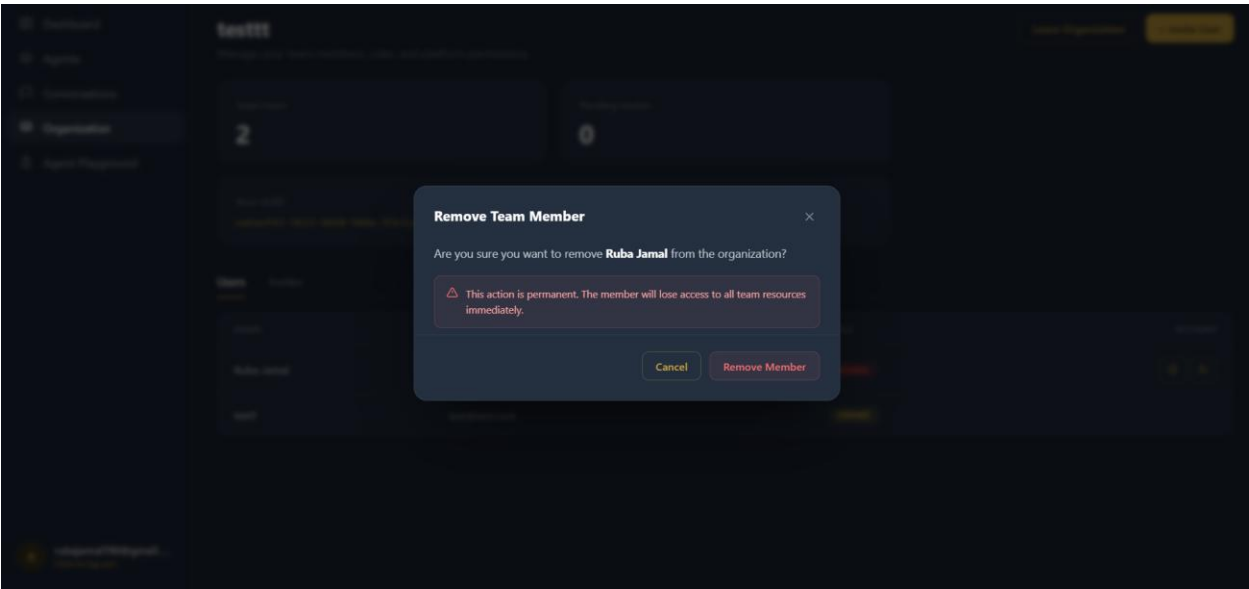


Figure A.9: Remove Team Member Page

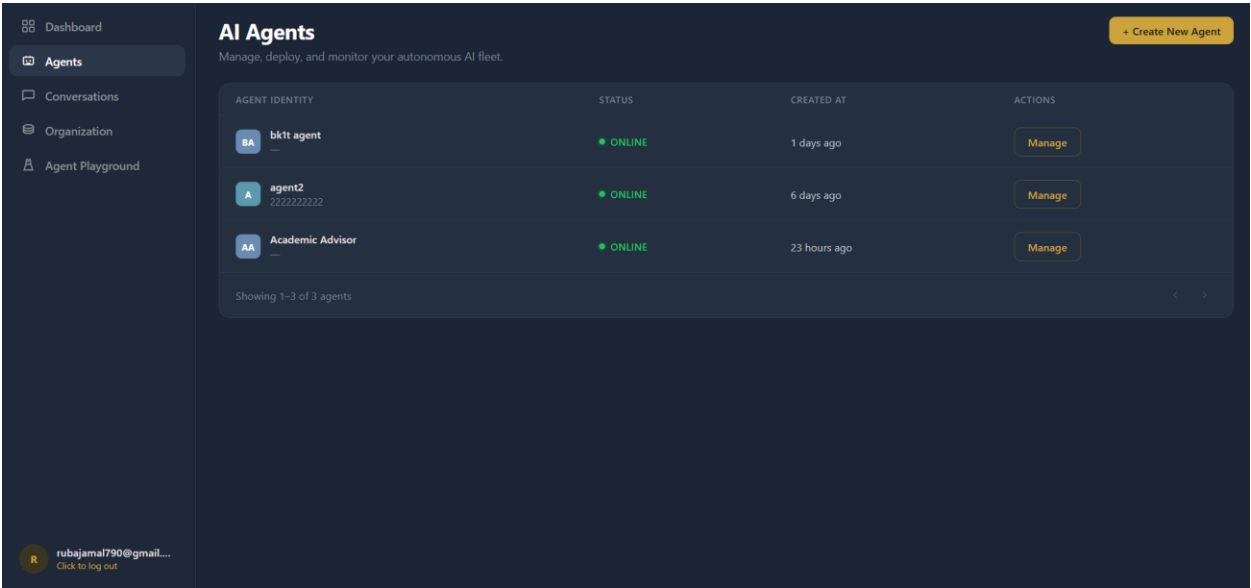


Figure A.10: View Agent Page

Create Agent Cancel Create Agent

Basic Identity

Agent Name: e.g. Customer Support

Secret Key: A strong shared secret

Description: A short summary shown in the agents list

System Prompt: Describe how the agent should behave. This is the LLM system instruction.

Knowledge Base + Upload KB

NAME	DESCRIPTION	STATUS	ACTIONS
Save the agent first to upload knowledge bases.			

Tools & Capabilities + Create Tool

TYPE	NAME	URL	ACTIONS
------	------	-----	---------

From the agent first to add tools

Figure A.11: Create New Agent Page

Add Knowledge Base ×

NAME: e.g. Product Catalog 2024

DESCRIPTION: Describe what this knowledge base contains...

PDF FILE: Click to upload or drag and drop. PDF only - processing may take a few seconds

Cancel Upload

Figure A.12: Add Knowledge Base Page

Create Tool

TOOL NAME
e.g. Web Search API

TYPE
External API

PROMPT
Describe to the LLM when and how to use this tool...

DATA (JSON)

```
{  
  "url": "https://api.example.com/endpoint",  
  "method": "GET",  
  "parameters": [  
    { "name": "q", "type": "string", "description": "Search query", "required": true }  
  ]  
}
```

Expected shape: { url, method, parameters: [{ name, type, description, required }] }

Cancel Create Tool

Figure A.13: Add Tool Page

Dashboard

Agents

Conversations

Organization

Agent Playground

Conversations Logs

Monitor real-time interactions and system logs for all deployed AI agents.

ID	TARGET AGENT	EXTERNAL USER	TIMESTAMP	ACTIONS
#537cebef	agent2	174972806922397@11d	2026-05-30 23:15:53	
#fbc52eb8	agent2	137829996863682@11d	2026-05-30 23:15:04	
#67826391	agent2	-	2026-05-26 20:26:42	
#489f8644	agent2	-	2026-05-26 14:52:27	
#8db26061	agent2	-	2026-05-26 14:38:14	
#e16ece3d	agent2	-	2026-05-26 14:34:56	
#63e05e68	agent2	-	2026-05-26 14:32:37	
#439b47aa	agent2	-	2026-05-26 14:32:06	

Showing 1 to 8 of 12 logs

rubajamal790@gmail...

Click to log out

Figure A.14: Conversation Logs View Page

Dashboard

Agents

Conversations

Organization

Agent Playground

AI Playground

Select Agent

rubat test

Configuration

agent secret key

Paste this agent's secret key

The playground authenticates against POST /chat using the agent's secret key (not your account JWT).

Send a message to chat with rubat test.

Paste an agent secret key to begin...

A secret key is required — paste your agent's secret_key above.

Figure A.15: Agent Playground Page